ACG + CLOUDILIC / APPLIED LEARNING SERIES

PYTHON, AI & AUTOMATION PRACTICAL FOUNDATIONS

Code literacy • Git + GitHub • SQL • Prompt Engineering • AI APIs • RAG • Local AI • Security • Automation

A practical program for someone who does not need to become a developer. The learner should be able to read and adapt Python, debug errors, work safely with Git and databases, use AI coding agents, connect software to AI APIs, understand RAG and local models, and turn useful scripts into repeatable or scheduled workflows.

7 SESSIONS / 3 HOURS EACH / WEEKLY PRACTICE

PROPOSED FIRST SESSION • THURSDAY 15 OCTOBER 2026 • 18:00-21:00

PROGRAM PRINCIPLE

Read → Understand → Debug → Integrate → Automate

Python is compressed into four mixed sessions. Each of those sessions pairs Python with a second high-value skill, so the learner builds technical literacy without spending the whole program on language syntax.

01 / COURSE POSITIONING

Technical confidence, not developer depth.

The program is built around practical understanding of existing systems: code, source control, databases, AI services, local models and automation. The learner uses AI as leverage but keeps human review and technical judgment.

“I do not need to write everything from scratch. I need to understand what I am looking at, know what an error means, make the change I need, and safely connect the pieces into a working result.”

THE PARTICIPANT WILL BE ABLE TO

READ & TRACE CODE

Recognize variables, conditions, functions, imports, files and APIs; explain what a script does and where data moves.

DEBUG WITH EVIDENCE

Read tracebacks/logs, isolate the failing line, form a hypothesis, make one change and verify the result.

USE GIT SAFELY

Clone a project, understand commits and branches, pull updates, inspect changes and recover from a bad edit.

QUERY BUSINESS DATA

Understand tables, records and IDs; use SELECT, WHERE, JOIN and GROUP BY for practical business-system questions.

CONNECT AI TO SOFTWARE

Use OpenAI/Claude-style APIs, structured JSON, tool/function calling, token/cost concepts and robust error handling.

AUTOMATE & SCHEDULE

Turn a reviewed script into a repeatable or scheduled task with logs, clear inputs/outputs and safe stop/recovery steps.

NOT THE GOAL

Advanced algorithms, framework-heavy application development, software architecture specialization, or memorizing syntax. The learner may rely on AI to produce code, but must understand enough to review diffs, test behavior, protect credentials/data and recover from mistakes.

02 / SEVEN-SESSION ROADMAP

Four mixed Python sessions, then integration depth.

Each 3-hour session is practical. In Sessions 1-4, roughly half the session is Python/code literacy and the other half adds a complementary capability. Sessions 5-7 integrate AI tooling, local models, RAG and real automation.

SESSION	PART A - PYTHON / CORE TECH	PART B - SECOND FOCUS
01	Python literacy: execution flow, variables, types, operators, reading short scripts line-by-line.	Git + GitHub basics: clone, status, commit, branch, pull/update, diff and safe recovery.
02	Python logic + debugging: if/else, loops, lists/dicts, tracebacks, exceptions and one-change-at-a-time fixes.	SQL & databases: tables/records/IDs, SELECT, WHERE, JOIN, GROUP BY and interpreting business data.
03	Python files + integration: functions, modules, paths, CSV/JSON, requests-style API calls and parsing responses.	Prompt Engineering: zero/few-shot, role/context, step-back, structured outputs, delimiters, variables and evaluation.
04	Python automation: reusable scripts, parameters, environment variables, logging, API wrappers and scheduler readiness.	AI API integration + security: system prompts, JSON schemas, tool/function calling, tokens/cost, retries, .env and secrets.
05	AI coding-agent workflow: write a specification, give context, inspect before modifying, review diffs, run tests and revert.	Claude Code + local AI ecosystem: online agent use, Hermes local models, model fit, switching and offline operation.
06	RAG / company-document AI: documents → chunks → embeddings → vector store → retrieval → grounded answer.	Automation & scheduling: approved recurring workflows, OS scheduler concepts, logs, checkpoints and failure handling.
07	Capstone integration: existing project + Git + database/file + AI + automation.	Review & handover: security check, documentation, restore/revert path, demo and next-step plan.

BETWEEN SESSIONS

Short homework on the actual laptop: explain one code path, fix one issue, use Git to checkpoint work, query a small dataset, improve one prompt, or turn one task into a repeatable run.

03 / CORE SKILLS & AI WORKFLOWS

From prompts to production-minded automation.

The new topics bridge basic Python literacy with the real systems the learner will encounter in Odoo, internal tools, AI-assisted workflows and local/private AI.

GIT + GITHUB - HIGH PRIORITY

- Clone/open a project; understand working tree vs committed history.
- Commit small checkpoints, create/switch branches and pull updates.
- Review diffs before accepting AI edits; revert or restore when something breaks.

SQL & DATABASES - HIGH PRIORITY

- Understand tables, rows, records, IDs, relationships and keys.
- Practice SELECT, WHERE, JOIN and GROUP BY on business-style examples.
- Connect SQL thinking to Odoo/reporting: “what data do I need, from where, and how is it related?”

AI API INTEGRATION

- OpenAI/Claude-style API flow: request → model → structured response.
- System prompts, JSON schemas, tool/function calling, tokens/cost and rate/error handling.
- Use environment variables; never paste API keys into source code.

PROMPT TECHNIQUES - ROADMAP.SH

- Zero-shot and one/few-shot examples; system, role and contextual prompting.
- Step-back / decomposition, structured output, delimiters and reusable variables/placeholders.
- Prompt injection awareness, output evaluation, prompt versioning and cost/latency trade-offs.

RAG / COMPANY-DOCUMENT AI

- Documents → chunks → embeddings → vector database → retrieve relevant context → LLM answer.
- Use cases: policies, manuals, product catalogs, support knowledge and selected Odoo/business data.
- Focus on grounding, permissions, freshness and verifying sources.

SECURITY + AGENT WORKFLOW

- .env files, credentials, permissions, backups and customer-data boundaries.
- Agent pattern: specify → provide context → inspect first → plan → modify → review diff → test → accept/revert.
- Never allow broad destructive actions or production automation without review and rollback.

ONLINE + LOCAL REALITY

Claude Code is used as an online coding agent. Hermes demonstrates local models that can run on the laptop after the runtime/model are downloaded. The learner should know when to choose cloud capability versus local privacy/availability.

Prompt topics aligned with roadmap.sh Prompt Engineering Roadmap • Local-model flow aligned with Hermes Agent documentation

04 / DELIVERY, ASSESSMENT & CAPSTONE

Success = confident use of existing code and connected systems.

Assessment is practical. The learner is judged on whether they can understand, troubleshoot, integrate and safely operate a useful workflow - not on writing a large application from a blank file.

DELIVERY PLAN

7 weekly sessions × 3 hours • Proposed first session: Thu 15 Oct 2026, 18:00-21:00 • Guided homework between sessions. Weekly spacing gives time to practice on the actual laptop and bring back real questions/errors.

PRACTICAL ASSESSMENT

20%	Read & explain code flow
15%	Debug from traceback/log evidence
15%	Git/GitHub safe workflow
15%	SQL/data understanding
15%	AI prompting + API integration
10%	Security & review discipline
10%	Automation / schedule readiness

FINAL CAPSTONE

Starting point: an existing small workflow - not a blank file.

1. Clone/open the project and create a safe Git checkpoint.
2. Explain the main Python flow and identify one issue.
3. Use traceback/log evidence to fix the issue.
4. Query or read business data (SQL or CSV/JSON).
5. Use a strong prompt/specification with an AI coding agent.
6. Integrate one AI API or structured model output.
7. Run one local Hermes model task and explain cloud vs local choice.
8. Add one useful automation/schedule with logs.
9. Document credentials, permissions, run/stop/revert and handover steps.

FIRST-SESSION READINESS

Laptop with Python 3 + editor/terminal; Git installed; sample repo; SQLite or simple SQL environment; Claude Code authenticated for online labs; Hermes installation checked with storage/hardware for a suitable local model; use sample data and non-production credentials.

FINAL PROGRAM OUTCOME

The learner can read and debug Python, work safely with Git, understand SQL-backed business data, write effective prompts, integrate AI APIs, reason about RAG and local models, supervise an AI coding agent, protect secrets/data, and turn a reviewed workflow into repeatable automation.